

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Electrónica y Automatización Asignatura: Fundamentos de Programación NRC: 28561
	Apellidos y nombres del estudiante: Figueroa Sofía Santiago Velasco Tigse Alexander

Taller 4

Laboratorio propuesto (entregable práctico) Objetivo: implementar el CRUD completo y probarlo con al menos 10 registros.

1. Crear el proyecto en Code:Blocks y pegar el código en un solo archivo main.c.
2. Ejecutar y agregar 3 estudiantes. Verificar que se cree estudiantes.txt y que cada estudiante quede en una línea.
3. Probar la validación de ID duplicado (intentar agregar el mismo ID dos veces).
4. Listar y verificar el formato de tabla en consola.
5. Consultar por ID existente y por uno inexistente. Fundamentos de Programación Página 9
6. Actualizar un estudiante y verificar que los cambios persistan al listar de nuevo.
7. Eliminar un estudiante y verificar que ya no aparece.
8. Abrir estudiantes.txt con un editor de texto y comprobar que el formato sigue siendo “;” por campos.

Código:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#define ARCHIVO_EST "estudiantes.txt"
```

```

typedef struct
{
    char id[20];
    char apellidos[50];
    char nombres[50];
    int edad;
} Estudiante;

/* Elimina el salto de linea que deja fgets */
void limpiarNuevaLinea(char cadena[])
{
    cadena[strcspn(cadena, "\n")] = '\0';
}

/* Lee una cadena de forma segura */
void leerCadena(const char mensaje[], char destino[], int tamano)
{
    printf("%s", mensaje);
    fgets(destino, tamano, stdin);
    limpiarNuevaLinea(destino);
}

/* Lee un numero entero */
int leerEntero(const char mensaje[])
{
    char cadena[50]
    printf("%s", mensaje);
    fgets(cadena, sizeof(cadena), stdin);
    return atoi(cadena);
}

/* Convierte una linea del archivo en una estructura Estudiante */

```

```

int parsearEstudiante(const char linea[], Estudiante *e)
{
    int campos;

    campos = sscanf(linea, "%19[^\n];%49[^\n];%49[^\n];%d",
        e->id,
        e->apellidos,
        e->nombres,
        &e->edad);

    return campos == 4;
}

```

/* Verifica si un ID ya existe */

```

int existeId(const char idBuscado[])
{
    FILE *archivo;
    char linea[200];
    Estudiante e;
    archivo = fopen(ARCHIVO_EST, "r");
    if (archivo == NULL)
    {
        return 0;
    }
    while (fgets(linea, sizeof(linea), archivo))
    {
        if (parsearEstudiante(linea, &e))
        {
            if (strcmp(e.id, idBuscado) == 0)
            {
                fclose(archivo);
                return 1;
            }
        }
    }
}

```

```

        }

    }

}

fclose(archivo);

return 0;

}

/* Agrega un estudiante */

void agregarEstudiante()

{

    FILE *archivo;

    Estudiante nuevo;

    printf("\n--- AGREGAR ESTUDIANTE ---\n");

    leerCadena("Ingrese ID: ", nuevo.id, sizeof(nuevo.id));

    if (existeId(nuevo.id))

    {

        printf("Error: ya existe un estudiante con ese ID.\n");

        return;

    }

    leerCadena("Ingrese apellidos: ",

                nuevo.apellidos,

                sizeof(nuevo.apellidos));

    leerCadena("Ingrese nombres: ",

                nuevo.nombres,

                sizeof(nuevo.nombres));

    nuevo.edad = leerEntero("Ingrese edad: ");

    archivo = fopen(ARCHIVO_EST, "a");

    if (archivo == NULL)

    {

        printf("Error: no se pudo abrir el archivo.\n");
    }
}

```

```

        return;
    }

    fprintf(archivo, "%s;%s;%s;%d\n",
            nuevo.id,
            nuevo.apellidos,
            nuevo.nombres,
            nuevo.edad);

    fclose(archivo);

    printf("Estudiante agregado correctamente.\n");
}

/* Lista todos los estudiantes */
void listarEstudiantes()
{
    FILE *archivo;
    char linea[200];
    Estudiante e;
    int cantidad = 0;

    archivo = fopen(ARCHIVO_EST, "r");
    if (archivo == NULL)
    {
        printf("\nNo existen estudiantes registrados.\n");
        return;
    }

    printf("\n%-12s %-20s %-20s %-5s\n",
            "ID", "APELLIDOS", "NOMBRES", "EDAD");
    printf("-----\n");
    while (fgets(linea, sizeof(linea), archivo))
    {
        if (parsearEstudiante(linea, &e))

```

```

    {
        printf("%-12s %-20s %-20s %-5d\n",
            e.id,
            e.apellidos,
            e.nombres,
            e.edad);
        cantidad++;
    }
}

fclose(archivo);

if (cantidad == 0)
{
    printf("No existen estudiantes registrados.\n");
}
}

/* Busca un estudiante por ID */

int buscarPorId(const char idBuscado[], Estudiante *resultado)
{
    FILE *archivo;
    char linea[200];
    Estudiante e;
    archivo = fopen(ARCHIVO_EST, "r");
    if (archivo == NULL)
    {
        return 0;
    }
    while (fgets(linea, sizeof(linea), archivo))
    {
        if (parsearEstudiante(linea, &e))

```

```

    {
        if (strcmp(e.id, idBuscado) == 0)
        {
            *resultado = e;
            fclose(archivo);
            return 1;
        }
    }
}

fclose(archivo);

return 0;
}

/* Consulta un estudiante por ID */

void consultarEstudiante()
{
    char idBuscado[20];

    Estudiante e;

    printf("\n--- CONSULTAR ESTUDIANTE ---\n");

    leerCadena("Ingrese el ID que desea buscar: ",
               idBuscado,
               sizeof(idBuscado));

    if (buscarPorId(idBuscado, &e))
    {
        printf("\nEstudiante encontrado:\n");
        printf("ID: %s\n", e.id);
        printf("Apellidos: %s\n", e.apellidos);
        printf("Nombres: %s\n", e.nombres);
        printf("Edad: %d\n", e.edad);
    }
}

```

```

else

{

    printf("No se encontro un estudiante con ese ID.\n");

}

}

/* Actualiza un estudiante */

void actualizarEstudiante()

{

    FILE *archivo;

    FILE *temporal;

    char linea[200];

    char idActualizar[20];

    Estudiante e;

    int actualizado = 0;

    printf("\n--- ACTUALIZAR ESTUDIANTE ---\n");

    leerCadena("Ingrese el ID que desea actualizar: ",

        idActualizar,

        sizeof(idActualizar));

    archivo = fopen(ARCHIVO_EST, "r");

    if (archivo == NULL)

    {

        printf("No existe el archivo de estudiantes.\n");

        return;

    }

    temporal = fopen("tmp.txt", "w");

    if (temporal == NULL)

    {

        fclose(archivo);

        printf("No se pudo crear el archivo temporal.\n");

```



```

    return;
}
while (fgets(linea, sizeof(linea), archivo))
{
    if (parsearEstudiante(linea, &e))
    {
        if (strcmp(e.id, idActualizar) == 0)
        {
            printf("\nIngrese los nuevos datos:\n");
            leerCadena("Nuevos apellidos: ",
                      e.apellidos,
                      sizeof(e.apellidos));
            leerCadena("Nuevos nombres: ",
                      e.nombres,
                      sizeof(e.nombres));
            e.edad = leerEntero("Nueva edad: ");
            actualizado = 1;
        }
        fprintf(temporal, "%s;%s;%s;%d\n",
                e.id,
                e.apellidos,
                e.nombres,
                e.edad);
    }
}
fclose(archivo);
fclose(temporal);
remove(ARCHIVO_EST);
rename("tmp.txt", ARCHIVO_EST);

```

```

if (actualizado)
{
    printf("Estudiante actualizado correctamente.\n");
}
else
{
    printf("No se encontro un estudiante con ese ID.\n");
}
}

/* Elimina un estudiante */

void eliminarEstudiante()
{
    FILE *archivo;
    FILE *temporal;
    char linea[200];
    char idEliminar[20];
    Estudiante e;
    int eliminado = 0;
    printf("\n--- ELIMINAR ESTUDIANTE ---\n");
    leerCadena("Ingrese el ID que desea eliminar: ",
        idEliminar,
        sizeof(idEliminar));
    archivo = fopen(ARCHIVO_EST, "r");
    if (archivo == NULL)
    {
        printf("No existe el archivo de estudiantes.\n");
        return;
    }
}

```

```
temporal = fopen("tmp.txt", "w");

if (temporal == NULL)
{
    fclose(archivo);

    printf("No se pudo crear el archivo temporal.\n");

    return;
}

while (fgets(linea, sizeof(linea), archivo))
{
    if (parsearEstudiante(linea, &e))
    {
        if (strcmp(e.id, idEliminar) != 0)
        {
            fprintf(temporal, "%s;%s;%s;%d\n",

                e.id,

                e.apellidos,

                e.nombres,

                e.edad);
        }
        else
        {
            eliminado = 1;
        }
    }
}

fclose(archivo);

fclose(temporal);

remove(ARCHIVO_EST);
```

```

rename("tmp.txt", ARCHIVO_EST);

if (eliminado)
{
    printf("Estudiante eliminado correctamente.\n");
}
else
{
    printf("No se encontro un estudiante con ese ID.\n");
}
}

/* Muestra el menu */

int mostrarMenu()
{
    int opcion;

    printf("\n===== \n");
    printf(" CRUD DE ESTUDIANTES\n");
    printf("===== \n");
    printf("1. Agregar estudiante\n");
    printf("2. Listar estudiantes\n");
    printf("3. Consultar estudiante por ID\n");
    printf("4. Actualizar estudiante\n");
    printf("5. Eliminar estudiante\n");
    printf("0. Salir\n");
    printf("===== \n");

    opcion = leerEntero("Seleccione una opcion: ");

    return opcion;
}

int main()

```

```
{  
  
    int opcion;  
  
    do  
  
    {  
  
        opcion = mostrarMenu();  
        switch (opcion)  
        {  
  
            case 1:  
  
                agregarEstudiante();  
  
                break;  
  
            case 2:  
  
                listarEstudiantes();  
  
                break;  
  
            case 3:  
  
                consultarEstudiante();  
  
                break;  
  
            case 4:  
  
                actualizarEstudiante();  
  
                break;  
  
            case 5:  
  
                eliminarEstudiante();  
  
                break;  
  
            case 0:  
  
                printf("\nSaliendo del programa...\n");  
  
                break;  
  
            default:  
  
                printf("\nOpcion no valida.\n");  
  
                break;  
  
        }  
    }  
}
```

```
} while (opcion != 0);
```

```
return 0;
```

```
}
```

The screenshot shows a C++ IDE with a file named 'Untitled1.c'. The code implements a student management system (CRUD) using a text file 'estudiantes.txt'. It defines a 'Estudiante' struct with fields for ID, last name, first name, and age. Functions include 'limpiarNuevaLinea' for clearing lines, 'leerCadena' for reading strings, 'leerEntero' for reading integers, and 'parsearEstudiante' for parsing a line from the file into a student struct. The main function uses a menu-driven loop to allow users to add, list, search, update, or delete students. The terminal output shows the program running, with the user entering '19' for age and selecting option '2' to list students. A table displays the current student data:

ID	APELLIDOS	NOMBRES	EDAD
100457027	figeroa	sofia	19

This is a duplicate of the previous screenshot, showing the same C++ code and terminal output for the student management program. The code and terminal output are identical to the one above.